

Arquitectura y Explicación del Código: Tareas React

Desglose conceptual del flujo de datos, inmutabilidad y renderizado reactivo con Vite y JSX.

1. Los Estados de Memoria (`useState`)

En React, las variables comunes se reinician en cada renderizado. Para persistir datos y provocar actualizaciones visuales automáticas cuando la información cambia, se declaran tres estados clave:

tareas

Array de objetos { `id`, `descripcion`, `hecho` }. Es la **única fuente de la verdad** para todas las tareas de la aplicación.

texto

String vinculado bidireccionalmente al `<input>`. Captura cada carácter ingresado por el usuario en tiempo real.

nextId

Contador numérico incremental (inicia en 5). Garantiza un identificador único para cada nueva tarea sin colisiones.

2. Validación Reactiva y Envío de Formulario

La consigna exige habilitar el botón a partir del tercer carácter y admitir un tope de 60, además de permitir la confirmación mediante la tecla Enter.

Mecanismo	Implementación	Explicación Técnica
Regla 3-60 caracteres	<code>texto.trim().length >= 3</code>	<code>trim()</code> elimina espacios vacíos periféricos para evitar que se creen tareas ficticias compuestas solo por barras espaciadoras.
Tope máximo	<code>maxLength={60}</code>	Atributo HTML nativo que restringe físicamente la entrada a un máximo de 60 caracteres directamente en el navegador.
Habilitación del botón	<code>disabled={!esValido}</code>	React evalúa <code>esValido</code> en cada pulsación. Al llegar al 3° carácter pasa a <code>true</code> y remueve el bloqueo del botón al instante.
Disparo con Enter y Clic	<code><form onSubmit={...}></code>	Aprovecha la semántica web nativa: cualquier botón de envío dentro de un formulario se activa tanto al hacer clic como al presionar Enter.

Ventaja Arquitectónica: Cero Event Listeners manuales

Al usar un `<form>` con `onSubmit`, no hace falta programar escuchadores complejos de teclado (`onKeyDown`) evaluando si `e.key === 'Enter'`. El navegador y React resuelven la accesibilidad de forma natural y estandarizada.

3. Las Funciones de Lógica y el Principio de Inmutabilidad

En React nunca se debe mutar el estado directamente (por ejemplo, usando `tareas.push()` o `tarea.hecho = true`). Toda modificación produce un nuevo array o un nuevo objeto mediante copias:

Función	Método / Operador	Comportamiento
handleAgregar	<code>[...tareas, nuevaTarea]</code>	Aplica el operador <i>spread</i> para clonar la lista actual y añadir la nueva tarea al final con <code>hecho: false</code> . Luego resetea el input con <code>setTexto('')</code> .
toggleHecho(id)	<code>tareas.map(...)</code>	Recorre el array. Si el elemento coincide con el ID seleccionado, crea una copia invirtiendo su valor: <code>{ ...t, hecho: !t.hecho }</code> . El resto permanece intacto.
eliminarRealizadas	<code>tareas.filter((t) => !t.hecho)</code>	Genera una nueva lista descartando las que posean <code>hecho: true</code> . Solo sobreviven aquellas cuyo estado sea pendiente.

4. Estado Único vs. Estado Duplicado (Derivación en Vuelo)

Un error muy habitual es declarar dos estados separados: uno para pendientes y otro para realizadas. Nuestra solución aplica **estado derivado**:

Filtros Dinámicos en Cada Render

```
const pendientes = tareas.filter(t => !t.hecho);
const realizadas = tareas.filter(t => t.hecho);
```

- 1 **Sincronización Total:** No existe riesgo de que una tarea quede duplicada en ambas tablas o huérfana en el limbo.
- 2 **Transición Inmediata:** Al marcar el checkbox, `toggleHecho` actualiza el booleano en `tareas`; React re-renderiza y la tarea salta automáticamente de la tabla "Pendientes" a la tabla "Realizadas".
- 3 **Persistencia Limpia:** Si en el futuro se desea almacenar las tareas en `localStorage` o enviarlas a un backend, solo se serializa un único array ordenado y consistente.

5. Checklist de Buenas Prácticas Aplicadas

- Prevención de recarga:** Uso estricto de `e.preventDefault()` al interceptar el submit.
- Keys en listas JSX:** Asignación rigurosa de `key={tarea.id}` en las filas `<tr>` para optimizar la reconciliación del Virtual DOM de React.
- Feedback visual de vacíos:** Empleo de mensajes condicionales cuando los arrays derivados quedan en longitud cero ("No hay tareas pendientes / realizadas").